

Lesson 7: Building a Complete Interactive App

Mission Brief

You've assembled every subsystem the Lumexa Explorer needs — variables and logic, a live dashboard, responsive controls, a live data feed, and a memory core that never forgets. Today is integration day: Mission Control needs you to combine every skill from Lessons 1–6 into one single, complete, working application, planned and built the way real engineers build real software — piece by piece, testing as you go.

Learning Objectives

By the end of this lesson, you will be able to:

- Plan an interactive app's data model and features before writing code
- Structure a small JavaScript app into clear, separated functions (state, rendering, storage, events)
- Combine DOM manipulation, event listeners, and Local Storage into one cohesive app
- Identify and fix bugs that appear only when features interact with each other
- Apply a "render from state" architecture, where the UI always reflects one source of truth

What You'll Build

A complete "Crew Task Tracker" app — a simplified but fully real and functional relative of the course's to-do app: add tasks, mark them complete, delete them, filter by all/active/completed, and persist everything in Local Storage — combining Lessons 2, 3, and 6 into a single working application from scratch.

Prerequisites

- Lesson 2 (DOM manipulation)
- Lesson 3 (event listeners)
- Lesson 6 (Local Storage)
- Comfort writing multiple functions that call each other

Key Concepts

- Planning a data model before coding
- "Single source of truth" state + re-render pattern
- Separating concerns: state functions, render functions, storage functions, event handlers
- Filtering an array with `.filter()`
- Integration bugs (bugs that only appear when features combine)

Concept Explanation

Every app you've built so far in this course has been small and focused on one concept. Real apps combine many concepts at once, and the way you **organize** your code becomes just as important as the code itself. A proven, simple architecture for small interactive apps is: keep one **array or object as the single source of truth** for all your data (the "state"), write one function that **renders** the entire UI based on that state, and make every user action (add, delete, toggle) simply update the state and then call the render function again — rather than manually patching individual DOM elements in ad-hoc ways.

This "render from state" pattern has a major benefit: your UI can never get out of sync with your data, because the UI is always rebuilt directly from the current state, every time anything changes. It costs a little performance (rebuilding a list is slightly more work than surgically updating one element), but for small apps like the ones in this course, that cost is irrelevant, and the simplicity is worth far more than the performance.

Before writing any code for a real app, professional developers **plan the data model**: what does one task look like as a JavaScript object? For a to-do-style app, a reasonable shape is `{ id, text, completed }`. The `id` matters more than it might seem — using an array *index* to identify which task to delete or toggle becomes unreliable once items are removed (indexes shift), so a stable unique `id` (even something simple like `Date.now()` at creation time) avoids a whole category of bugs.

Filtering (all/active/completed) is a great use for the array method `.filter()`, which returns a *new* array containing only the elements that pass a test function — for example, `tasks.filter(task => !task.completed)` returns only incomplete tasks, without modifying the original array. Combining this with the render function means "switch filter" is just: change a `currentFilter` variable, then re-render using the filtered subset.

Finally, **integration bugs** are a normal and expected part of combining features — a delete button might accidentally trigger a toggle because of event delegation catching the wrong target, or a re-render might wipe out an input field's typed-but-unsaved text. The discipline of testing each feature individually, then testing them *together* deliberately (add a task, then filter, then delete, then reload the page, in that order) is how professionals catch these before shipping.

Guided Coding

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Crew Task Tracker</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; max-
width: 480px; }
    #task-list { list-style: none; padding: 0; }
    #task-list li { display: flex; align-items: center; gap: 0.5rem; padding: 0.4rem 0; }
    #task-list li.completed span { text-decoration: line-through; color: #8b93b0; }
    .filters button.active { font-weight: bold; text-decoration: underline; }
    .delete-btn { margin-left: auto; }
  </style>
</head>
<body>
  <h1>Crew Task Tracker</h1>
  <input type="text" id="task-input" placeholder="New task..." />
  <button id="add-btn">Add Task</button>

  <div class="filters">
    <button data-filter="all" class="active">All</button>
    <button data-filter="active">Active</button>
    <button data-filter="completed">Completed</button>
  </div>

  <ul id="task-list"></ul>
  <script src="script.js"></script>
</body>
</html>

```

```

// script.js

const STORAGE_KEY = "lumexa-crew-tasks";

// --- ELEMENTS ---
const taskInput = document.querySelector("#task-input");
const addButton = document.querySelector("#add-btn");
const taskListEl = document.querySelector("#task-list");
const filterButtons = document.querySelectorAll(".filters button");

// --- STATE (single source of truth) ---
let tasks = loadTasks();
let currentFilter = "all";

// --- STORAGE FUNCTIONS ---
function loadTasks() {
  const raw = localStorage.getItem(STORAGE_KEY);
  if (raw === null) return [];
  try {
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch (error) {
    console.error("Corrupted task data, resetting.", error);
    return [];
  }
}

function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

// --- STATE-CHANGING FUNCTIONS ---
function addTask(text) {
  tasks.push({ id: Date.now(), text: text, completed: false });
  saveTasks();
  render();
}

function toggleTask(id) {
  tasks = tasks.map((task) =>
    task.id === id ? { ...task, completed: !task.completed } : task
  );
  saveTasks();
  render();
}

function deleteTask(id) {
  tasks = tasks.filter((task) => task.id !== id);
}

```

```

    saveTasks();
    render();
}

function setFilter(filterName) {
    currentFilter = filterName;
    render();
}

// --- RENDER (rebuilds the UI entirely from current state) ---
function getVisibleTasks() {
    if (currentFilter === "active") return tasks.filter((task) => !task.completed);
    if (currentFilter === "completed") return tasks.filter((task) => task.completed);
    return tasks; // "all"
}

function render() {
    // Rebuild the task list
    taskListEl.innerHTML = "";
    const visibleTasks = getVisibleTasks();

    visibleTasks.forEach((task) => {
        const li = document.createElement("li");
        li.dataset.id = task.id; // store the id on the element for event delegation
        if (task.completed) li.classList.add("completed");

        const checkbox = document.createElement("input");
        checkbox.type = "checkbox";
        checkbox.checked = task.completed;
        checkbox.classList.add("toggle-checkbox");

        const label = document.createElement("span");
        label.textContent = task.text;

        const deleteBtn = document.createElement("button");
        deleteBtn.textContent = "Delete";
        deleteBtn.classList.add("delete-btn");

        li.append(checkbox, label, deleteBtn);
        taskListEl.appendChild(li);
    });

    // Update which filter button looks "active"
    filterButtons.forEach((btn) => {
        btn.classList.toggle("active", btn.dataset.filter === currentFilter);
    });
}

// --- EVENTS ---

```

```

addButton.addEventListener("click", () => {
  const text = taskInput.value.trim();
  if (text === "") return;
  addTask(text);
  taskInput.value = "";
});

taskInput.addEventListener("keydown", (event) => {
  if (event.key === "Enter") addButton.click();
});

// Event delegation: one listener handles toggle + delete for every task,
// including tasks added after the page first loaded.
taskListEl.addEventListener("click", (event) => {
  const li = event.target.closest("li");
  if (!li) return;
  const id = Number(li.dataset.id);

  if (event.target.classList.contains("toggle-checkbox")) {
    toggleTask(id);
  } else if (event.target.classList.contains("delete-btn")) {
    deleteTask(id);
  }
});

filterButtons.forEach(btn => {
  btn.addEventListener("click", () => setFilter(btn.dataset.filter));
});

// --- INITIAL RENDER ON PAGE LOAD ---
render();

```

Code Walkthrough

- **State:** `tasks` (loaded immediately from storage) and `currentFilter` are the *only* two variables that determine everything the app shows. Every other function either reads them (`render`) or updates them (`add/toggle/delete/setFilter`).
- **loadTasks/saveTasks:** identical pattern to Lesson 6, isolated into their own functions so storage logic is never duplicated.
- **addTask/toggleTask/deleteTask:** each one modifies `tasks`, immediately calls `saveTasks()` to persist, then calls `render()` to reflect the change — the consistent three-step pattern that keeps state, storage, and UI all in sync.
- **toggleTask** uses `.map()` with a spread (`{ ...task, completed: !task.completed }`) to build a *new* task object rather than mutating the old one directly — a safer pattern that avoids subtle bugs with shared object references.

- **getVisibleTasks/render:** `render()` never tries to update one specific `<li>` — it clears the whole list and rebuilds it from `getVisibleTasks()`, guaranteeing the screen always matches the current state and filter exactly.
- **Event delegation:** the click listener lives on the parent `#task-list`, uses `event.target.closest("li")` to find which task row was clicked (working correctly even if the checkbox or delete button itself was the actual target), and reads the task's `id` from a `data-id` attribute stored on the `<li>` — this is what lets brand-new tasks (added long after page load) work correctly with zero extra listener code.

Student Mission

1. Add an "edit" feature: double-clicking a task's text turns it into an editable `<input>` pre-filled with the current text; pressing Enter or blurring the input saves the new text back into `tasks` and re-renders.
2. Add a task counter, e.g., "3 tasks left", computed live from `tasks.filter(t => !t.completed).length`, displayed above the list.
3. Add a "Clear Completed" button that removes all completed tasks at once using `.filter()`.

Challenge

Add drag-to-reorder: allow the user to drag a task above/below another to change its order in the `tasks` array (research the `dragstart` / `dragover` / `drop` events), persisting the new order to Local Storage. This is a genuinely advanced feature — attempt it only after the core app works perfectly.

Experiment/Extension

Deliberately introduce a bug: change `toggleTask` to modify `task.completed` directly instead of using `.map()` with a spread, e.g., loop and mutate in place, then observe whether the bug is visible or not with your current test cases. Understanding *why* both approaches often look identical in simple cases, but diverge in trickier ones (shared references, undo features, etc.), builds real engineering judgment.

Common Mistakes

- Trying to update the DOM directly inside `addTask` / `toggleTask` / `deleteTask` instead of calling a single shared `render()` — leads to the UI and state drifting out of sync over time.
- Using array index instead of a stable `id` to identify which task to toggle/delete, which breaks as soon as the list is filtered or reordered.
- Forgetting to call `saveTasks()` after every state change, causing some actions to "not survive" a page reload while others do.
- Attaching individual click listeners to each task's delete button at render time instead of using delegation — technically works, but silently piles up duplicate listeners every time `render()` runs, since old `<li>` elements are destroyed and recreated each time.

- Not testing combinations: an "Add" that works, a "Delete" that works, and a "Filter" that works individually, but adding-then-filtering-then-reloading reveals a bug that individual testing missed.

Debugging Tips

- Use `console.log(tasks)` inside `render()` temporarily to confirm the state array actually looks the way you expect at every step of testing.
- If clicking a checkbox/delete button does nothing, verify with `console.log(event.target)` inside the delegated handler that you're correctly identifying which element was clicked.
- Use DevTools' Application tab to inspect the raw stored JSON directly and confirm it updates immediately after each action, not just after a reload.
- When a bug only appears after combining features, isolate it: comment out the filter feature temporarily and confirm add/delete/toggle still work correctly on their own, to narrow down where the bug actually lives.

Check Your Understanding

1. What does "single source of truth" mean in the context of this app's `tasks` array?
2. Why does `render()` rebuild the entire list instead of updating one `<li>` at a time?
3. Why is using a task's `id` safer than using its array index to identify it for deletion?
4. What does `.filter()` return, and does it modify the original array?
5. Give one example of an "integration bug" that could occur only once two features are combined.

Mini Quiz + Answer Key

Q1. What are the two "state" variables that fully determine what this app renders? A) `taskInput` and `addButton`
 B) `tasks` and `currentFilter` C) `STORAGE_KEY` and `render` D) None, state isn't used here

Q2. What does `tasks.filter(task => !task.completed)` return? A) The original array, mutated B) A new array containing only incomplete tasks C) A single task object D) `undefined`

Q3. Why is a task's `id` generated with `Date.now()` rather than using its position in the array? A) It's faster to type B) Array positions shift when items are added/removed/filtered, making them unreliable as identifiers C) `Date.now()` is required by Local Storage D) There is no real reason

Q4. What is the benefit of the "render from state" pattern? A) It's the fastest possible way to update the DOM B) The UI can never drift out of sync with the data, since it's always rebuilt from it C) It avoids needing event listeners D) It removes the need for Local Storage

Answer Key: 1-B, 2-B, 3-B, 4-B

Lesson Recap

You combined DOM manipulation, event listeners (including delegation), and Local Storage into one complete, working application, using a "single source of truth plus render function" architecture that keeps a real app's UI and data reliably in sync — the exact structure professional small apps are built with.

Homework

Add a "priority" field (`low` / `medium` / `high`) to each task, settable via a `<select>` shown next to the input when adding a task, displayed as a colored label on each rendered task, and persisted through the same load/save functions.

Portfolio Project Connection

This lesson is the direct blueprint for **Project 04, the Interactive To-Do App** — its full add/edit/delete/complete/filter/persist feature set is exactly what you practiced building here, just with an additional "edit" feature layered on top.